FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

# CS0065
# (INTELLIGENT SYSTEMS)

## TECHNICAL ASSESSMENT

# 3

## CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

| | |
|---|---|
| **Student Name** | Ingal and Bautista |
| **Section:** | An-43 |
| **Professor:** | Ms Tan |
| **Date:** | |

### I. BACKGROUND INFORMATION

**Case-Based Reasoning (CBR)** is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

## II. TECHNICAL ASSESSMENT

### CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

***Objective***

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

**Materials Needed**

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

**General Instructions**

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   *(Example: TA2_Patilano_Loverez.pdf)*
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

**1. Form a pair**

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

**2. Choose ONE (1) case study**

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

**Case Study Options:**

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

**3. Define your case base**

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

**4. Implement similarity assessment**

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

**5. Adapt the solution**

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

**6. Demonstrate learning (retain stage)**

After generating a final solution, store the new case into your case base.

**Your program must show:**

✓ "New case saved" or similar confirmation

✓ Updated case list

**7. Display clear program outputs**

Your output must include:

- New problem description

- Retrieved similar case
- Adapted final solution
- Updated case base

**8. Submission Requirements**

Submit the following:

**A. Program File**

- Python

**B. Short Documentation (1–2 pages)**

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

**C. Pair Reflection (5–7 sentences)**

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

**Case Studies:**

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

**Learning Outcome:**

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

**Scenario:**

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

**Learning:**
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

| Case Study 3: Intelligent Tutoring System |
|---|
| A CBR system identifies the learning struggles of students.<br>Student submits a math solution with repeated errors.<br>System retrieves past students with similar error patterns (e.g., misusing formulas).<br>It adapts feedback and personalized lessons based on successful interventions.<br>**Learning:**<br>System improves by retaining new student profiles and outcomes. |

**Your Code - Laboratory Tasks** ⇩

## A. Program File (disaster_cbr_dss.py)

Our program is written in Python 3 using only built-in modules (csv, math, os, sys). Run python disaster_cbr_dss.py for the demo event, python disaster_cbr_dss.py --new to type a new event, or --reset to restore the five seed cases. The case base is saved in disaster_cases.csv, so it can also be opened in Excel.

```
import csv
import math
import os
import sys


CSV_FILE = "disaster_cases.csv"



# ======================================================================
# 1. CASE REPRESENTATION
# ======================================================================
class DisasterCase:


    PROBLEM_FIELDS = ["hazard", "setting", "signal_or_magnitude",
                      "flood_depth_m", "population_at_risk", "hours_before_impact"]
    SOLUTION_FIELDS = ["evacuees", "shelters", "relief_packs",
                       "rescue_teams", "medical_units", "strategy"]


    def __init__(self, name, problem, solution=None):
        self.name = name
        self.problem = problem          # dict of PROBLEM_FIELDS
        self.solution = solution or {}  # dict of SOLUTION_FIELDS


    def to_row(self):
        return {"name": self.name, **self.problem, **self.solution}


    @classmethod
    def from_row(cls, row):
        p = {
            "hazard": row["hazard"],
            "setting": row["setting"],
            "signal_or_magnitude": float(row["signal_or_magnitude"]),
```

```
            "flood_depth_m": float(row["flood_depth_m"]),
            "population_at_risk": int(row["population_at_risk"]),
            "hours_before_impact": int(row["hours_before_impact"]),
        }
        s = {
            "evacuees": int(row["evacuees"]),
            "shelters": int(row["shelters"]),
            "relief_packs": int(row["relief_packs"]),
            "rescue_teams": int(row["rescue_teams"]),
            "medical_units": int(row["medical_units"]),
            "strategy": row["strategy"],
        }
        return cls(row["name"], p, s)



# Starting case base (5 past events)
SEED_CASES = [
    DisasterCase("Typhoon Ursula 2019",
                 {"hazard": "typhoon", "setting": "coastal", "signal_or_magnitude": 3,
                  "flood_depth_m": 0.5, "population_at_risk": 80000, "hours_before_impact": 48},
                 {"evacuees": 16000, "shelters": 40, "relief_packs": 48000, "rescue_teams": 12,
                  "medical_units": 6, "strategy": "Pre-emptive evacuation of shoreline barangays"}),
    DisasterCase("Typhoon Odette 2021",
                 {"hazard": "typhoon", "setting": "urban", "signal_or_magnitude": 4,
                  "flood_depth_m": 1.2, "population_at_risk": 250000, "hours_before_impact": 36},
                 {"evacuees": 62000, "shelters": 130, "relief_packs": 186000, "rescue_teams": 40,
                  "medical_units": 20, "strategy": "Forced evacuation of low-lying zones; schools as shelters"}),
    DisasterCase("Typhoon Kristine 2024",
                 {"hazard": "typhoon", "setting": "urban", "signal_or_magnitude": 3,
                  "flood_depth_m": 2.0, "population_at_risk": 300000, "hours_before_impact": 24},
                 {"evacuees": 90000, "shelters": 180, "relief_packs": 270000, "rescue_teams": 70,
                  "medical_units": 28, "strategy": "Rooftop and boat rescue; vertical evacuation to upper
floors"}),
    DisasterCase("Habagat Flood 2023",
                 {"hazard": "flood", "setting": "urban", "signal_or_magnitude": 0,
                  "flood_depth_m": 1.5, "population_at_risk": 150000, "hours_before_impact": 12},
                 {"evacuees": 35000, "shelters": 70, "relief_packs": 105000, "rescue_teams": 30,
                  "medical_units": 10, "strategy": "Boat rescue along riverbanks; gyms used as shelters"}),
    DisasterCase("Taal Volcano 2020",
                 {"hazard": "volcano", "setting": "rural", "signal_or_magnitude": 4,
                  "flood_depth_m": 0.0, "population_at_risk": 120000, "hours_before_impact": 6},
                 {"evacuees": 45000, "shelters": 90, "relief_packs": 135000, "rescue_teams": 20,
                  "medical_units": 15, "strategy": "Total evacuation of 14-km danger zone; N95 masks
distributed"}),
]



class CaseBase:
    """Loads and saves cases from/to a CSV file."""


    def __init__(self, path=CSV_FILE):
        self.path = path
        if not os.path.exists(path):
            self.reset()
        self.cases = self.load()


    def reset(self):
        self.save_all(SEED_CASES)


    def load(self):
        with open(self.path, newline="", encoding="utf-8") as f:
            return [DisasterCase.from_row(r) for r in csv.DictReader(f)]


    def save_all(self, cases):
```

```
        fields = ["name"] + DisasterCase.PROBLEM_FIELDS + DisasterCase.SOLUTION_FIELDS
        with open(self.path, "w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=fields)
            w.writeheader()
            for c in cases:
                w.writerow(c.to_row())

    def add(self, case):
        self.cases.append(case)
        self.save_all(self.cases)



# ======================================================================
# 2. SIMILARITY ASSESSMENT & RETRIEVAL
# ======================================================================
# How similar two hazard types are (1.0 = same)
HAZARD_SIM = {
    ("typhoon", "flood"): 0.7,    # typhoons often cause floods
    ("typhoon", "volcano"): 0.1,
    ("flood", "volcano"): 0.0,
}
# How similar two settings are
SETTING_SIM = {("urban", "coastal"): 0.5, ("urban", "rural"): 0.3, ("coastal", "rural"): 0.5}

# Gaussian "spread" for numeric features: a difference equal to the
# spread gives similarity of about 0.61; bigger differences drop fast.
SPREAD = {"signal_or_magnitude": 1.5, "flood_depth_m": 1.0,
          "population_at_risk": 150000, "hours_before_impact": 24}

WEIGHTS = {"hazard": 0.25, "setting": 0.15, "signal_or_magnitude": 0.15,
           "flood_depth_m": 0.20, "population_at_risk": 0.15, "hours_before_impact": 0.10}



def table_sim(table, a, b):
    if a == b:
        return 1.0
    return table.get((a, b), table.get((b, a), 0.0))



def gaussian_sim(a, b, spread):
    return math.exp(-((a - b) ** 2) / (2 * spread ** 2))



def local_sim(feature, a, b):
    if feature == "hazard":
        return table_sim(HAZARD_SIM, a, b)
    if feature == "setting":
        return table_sim(SETTING_SIM, a, b)
    return gaussian_sim(a, b, SPREAD[feature])



def similarity(new, old):
    """Global similarity = sum of weight x local similarity (weights add to 1)."""
    return sum(w * local_sim(f, new[f], old[f]) for f, w in WEIGHTS.items())



def retrieve(new_problem, case_base):
    scored = [(similarity(new_problem, c.problem), c) for c in case_base.cases]
    return sorted(scored, key=lambda t: t[0], reverse=True)
```

```
# ======================================================================
# 3. CASE ADAPTATION
# Step A: blend the TOP-2 cases. Each resource is converted to a
#         per-1,000-people rate, and the two rates are averaged using
#         the similarity scores as weights.
# Step B: multiply by the new population and by a severity index.
# Step C: pick the strategy of the best case, plus rule-based notes.
# ======================================================================
def severity_index(p):
    """Rough 0.8 - 1.4 multiplier from signal level, flood depth, and lead time."""
    s = 1.0
    s += 0.05 * max(0, p["signal_or_magnitude"] - 3)
    s += 0.10 * max(0, p["flood_depth_m"] - 1.0)
    if p["hours_before_impact"] < 24:
        s += 0.10                       # less time = more people stranded
    return round(min(max(s, 0.8), 1.4), 2)



def adapt(new_problem, top2):
    (s1, c1), (s2, c2) = top2
    w1, w2 = s1 / (s1 + s2), s2 / (s1 + s2)
    sev = severity_index(new_problem)
    pop_k = new_problem["population_at_risk"] / 1000
    notes = [f"Blend weights: {c1.name} = {w1:.2f}, {c2.name} = {w2:.2f}",
             f"Severity index = {sev}"]


    adapted = {}
    for field in ["evacuees", "shelters", "relief_packs", "rescue_teams", "medical_units"]:
        rate1 = c1.solution[field] / (c1.problem["population_at_risk"] / 1000)
        rate2 = c2.solution[field] / (c2.problem["population_at_risk"] / 1000)
        blended_rate = w1 * rate1 + w2 * rate2
        adapted[field] = math.ceil(round(blended_rate * pop_k * sev, 6))
        notes.append(f"{field:<14} rate/1k: {rate1:7.2f} & {rate2:7.2f} -> "
                     f"{blended_rate:7.2f} x {pop_k:.0f}k x {sev} = {adapted[field]:,}")


    # Strategy adaptation rules
    strategy = c1.solution["strategy"]
    extra = []
    low = strategy.lower()              # avoid repeating what is already there
    if new_problem["flood_depth_m"] >= 1.5 and "boat" not in low:
        extra.append("deploy rubber boats to flood-prone streets")
    if new_problem["hours_before_impact"] < 24:
        extra.append("start evacuation IMMEDIATELY (less than 24 hrs lead time)")
    if new_problem["setting"] == "urban" and "school" not in low:
        extra.append("open schools and covered courts as shelters")
    if extra:
        strategy += "; " + "; ".join(extra)
    adapted["strategy"] = strategy
    return adapted, notes



# ======================================================================
# 4. LEARNING (RETAIN)
# ======================================================================
def retain(case_base, name, problem, solution):
    case_base.add(DisasterCase(name, problem, solution))



# ======================================================================
# Display helpers and input
```

```
# ======================================================================
def hr(ch="-"):
    print(ch * 76)



def show_cases(case_base):
    print(f"   {'#':<3}{'Event':<26}{'Hazard':<9}{'Setting':<9}{'Depth':>6}{'Pop. at risk':>14}")
    for i, c in enumerate(case_base.cases, 1):
        p = c.problem
        print(f"   {i:<3}{c.name[:25]:<26}{p['hazard']:<9}{p['setting']:<9}"
              f"{p['flood_depth_m']:>5}m{p['population_at_risk']:>14,}")



def demo_event():
    return "Incoming Urban Typhoon", {
        "hazard": "typhoon", "setting": "urban", "signal_or_magnitude": 4,
        "flood_depth_m": 1.8, "population_at_risk": 200000, "hours_before_impact": 18}



def typed_event():
    print("Describe the new event:")
    name = input("   Event name: ") or "New Event"
    p = {"hazard": input("   Hazard (typhoon/flood/volcano): ").strip().lower(),
         "setting": input("   Setting (urban/coastal/rural): ").strip().lower(),
         "signal_or_magnitude": float(input("   Signal no. / alert level: ")),
         "flood_depth_m": float(input("   Expected flood depth (m): ")),
         "population_at_risk": int(input("   Population at risk: ")),
         "hours_before_impact": int(input("   Hours before impact: "))}
    return name, p



# ======================================================================
# MAIN - CBR cycle
# ======================================================================
def main():
    cb = CaseBase()
    if "--reset" in sys.argv:
        cb.reset()
        print("Case base reset to 5 seed cases.")
        return


    hr("=")
    print("  DISASTER RESPONSE DSS  -  Case-Based Reasoning (CDRRMO planning tool)")
    hr("=")
    print(f"Case base loaded from {CSV_FILE}: {len(cb.cases)} cases")
    show_cases(cb)


    name, problem = typed_event() if "--new" in sys.argv else demo_event()
    hr()
    print(f"NEW PROBLEM: {name}")
    for k, v in problem.items():
        print(f"   {k:<22}: {v:,}" if isinstance(v, int) else f"   {k:<22}: {v}")


    # ---- Retrieve ----
    hr()
    print("SIMILARITY ASSESSMENT & RETRIEVAL")
    ranked = retrieve(problem, cb)
    for rank, (score, c) in enumerate(ranked, 1):
        print(f"   Rank {rank}: {c.name:<24} similarity = {score:.3f}")
    top2 = ranked[:2]
    print(f"   >> Retrieved: {top2[0][1].name} and {top2[1][1].name}")
```

```
    # ---- Adapt ----
    hr()
    print("CASE ADAPTATION (blend of top-2 cases)")
    solution, notes = adapt(problem, top2)
    for n in notes:
        print(f"   {n}")


    hr()
    print("ADAPTED FINAL SOLUTION")
    for k in DisasterCase.SOLUTION_FIELDS:
        v = solution[k]
        print(f"   {k:<14}: {v:,}" if isinstance(v, int) else f"   {k:<14}: {v}")

    # ---- Retain ----
    retain(cb, name, problem, solution)
    hr()
    print(f"LEARNING: New case saved -> '{name}' written to {CSV_FILE}")
    print(f"Updated case base ({len(cb.cases)} cases):")
    show_cases(cb)
    hr("=")



if __name__ == "__main__":
    main()
```

## B. Short Documentation

### Title of Chosen Case Study

Case Study 1: Disaster Response Decision Support System (DSS) — a planning tool for a city disaster office (CDRRMO) that estimates evacuees, shelters, relief packs, rescue teams, and medical units for an incoming hazard by reusing what was done in similar past events.

### Description of the Case Base

Each case is a DisasterCase object with a problem part (hazard, setting, signal number or alert level, expected flood depth, population at risk, and hours before impact) and a solution part (evacuees, shelters, relief packs, rescue teams, medical units, and strategy). The five starting cases include typhoons, a monsoon flood, and a volcanic eruption, so the system can tell related and unrelated hazards apart. The values are simplified sample data. All cases are stored in a CSV file, where each row is one case.

| # | Past event | Hazard | Setting | Flood depth | Pop. at risk |
|---|---|---|---|---|---|
| 1 | Typhoon Ursula 2019 | typhoon | coastal | 0.5 m | 80,000 |
| 2 | Typhoon Odette 2021 | typhoon | urban | 1.2 m | 250,000 |
| 3 | Typhoon Kristine 2024 | typhoon | urban | 2.0 m | 300,000 |
| 4 | Habagat Flood 2023 | flood | urban | 1.5 m | 150,000 |
| 5 | Taal Volcano 2020 | volcano | rural | 0 m | 120,000 |

### Explanation of Similarity Method

We used weighted similarity with two kinds of local measures. Hazard and setting use a similarity table, for example typhoon vs. flood = 0.7 because typhoons usually bring floods, while typhoon vs. volcano = 0.1. Numeric features use a Gaussian similarity, $e^{(-d^2 / 2\sigma^2)}$, where $\sigma$ is a spread value per feature (1 m for flood depth, 150,000 for population). This makes small differences barely matter while large differences drop the score quickly. The local scores are multiplied by weights that add up to 1 (hazard 0.25, flood depth 0.20, setting, signal and population 0.15 each, lead time 0.10). For the demo urban typhoon, Odette (0.934) and Kristine (0.933) were the top two, while Taal Volcano scored only 0.478.

### Adaptation Logic

Instead of copying only the best case, our program blends the top two cases. Each resource is first changed into a rate per 1,000 people at risk, then the two rates are averaged using their similarity scores as weights. The blended rate is multiplied by the new population (in thousands) and by a severity index that goes up with a higher signal number, deeper floods, and less than 24 hours of warning (1.23 for the demo). For the strategy, the best case's strategy is kept and rules add steps only if they are missing, such as deploying boats when flood depth is at least 1.5 m or starting evacuation immediately when lead time is short. The final solution is saved as a new row in the CSV file.

## Screenshots of Program Execution

Run: disaster_cbr_dss

```
========================================================================
  DISASTER RESPONSE DSS  -  Case-Based Reasoning (CDRRMO planning tool)
========================================================================
Case base loaded from disaster_cases.csv: 5 cases
   #  Event                    Hazard   Setting   Depth  Pop. at risk
   1  Typhoon Ursula 2019      typhoon  coastal    0.5m        80,000
   2  Typhoon Odette 2021      typhoon  urban      1.2m       250,000
   3  Typhoon Kristine 2024    typhoon  urban      2.0m       300,000
   4  Habagat Flood 2023       flood    urban      1.5m       150,000
   5  Taal Volcano 2020        volcano  rural      0.0m       120,000
------------------------------------------------------------------------
NEW PROBLEM: Incoming Urban Typhoon
   hazard               : typhoon
   setting              : urban
   signal_or_magnitude  : 4
   flood_depth_m        : 1.8
   population_at_risk   : 200,000
   hours_before_impact  : 18
------------------------------------------------------------------------
SIMILARITY ASSESSMENT & RETRIEVAL
   Rank 1: Typhoon Odette 2021      similarity = 0.934
   Rank 2: Typhoon Kristine 2024    similarity = 0.933
   Rank 3: Habagat Flood 2023       similarity = 0.759
   Rank 4: Typhoon Ursula 2019      similarity = 0.686
   Rank 5: Taal Volcano 2020        similarity = 0.478
   >> Retrieved: Typhoon Odette 2021 and Typhoon Kristine 2024
------------------------------------------------------------------------
CASE ADAPTATION (blend of top-2 cases)
   Blend weights: Typhoon Odette 2021 = 0.50, Typhoon Kristine 2024 = 0.50
   Severity index = 1.23
   evacuees       rate/1k:  248.00 &  300.00 ->  273.98 x 200k x 1.23 = 67,400
   shelters       rate/1k:    0.52 &    0.60 ->    0.56 x 200k x 1.23 = 138
   relief_packs   rate/1k:  744.00 &  900.00 ->  821.95 x 200k x 1.23 = 202,200
   rescue_teams   rate/1k:    0.16 &    0.23 ->    0.20 x 200k x 1.23 = 49
   medical_units  rate/1k:    0.08 &    0.09 ->    0.09 x 200k x 1.23 = 22
------------------------------------------------------------------------
```

*Figure 2. Demo run — case base, new urban typhoon, and similarity ranking*

Run: disaster_cbr_dss (continued)

```
ADAPTED FINAL SOLUTION
   evacuees      : 67,400
   shelters      : 138
   relief_packs  : 202,200
   rescue_teams  : 49
   medical_units : 22
   strategy      : Forced evacuation of low-lying zones; schools as shelters; deploy rubber boats to flood-prone
streets; start evacuation IMMEDIATELY (less than 24 hrs lead time)
------------------------------------------------------------------------
LEARNING: New case saved -> 'Incoming Urban Typhoon' written to disaster_cases.csv
Updated case base (6 cases):
   #  Event                    Hazard   Setting   Depth  Pop. at risk
   1  Typhoon Ursula 2019      typhoon  coastal    0.5m        80,000
   2  Typhoon Odette 2021      typhoon  urban      1.2m       250,000
   3  Typhoon Kristine 2024    typhoon  urban      2.0m       300,000
   4  Habagat Flood 2023       flood    urban      1.5m       150,000
   5  Taal Volcano 2020        volcano  rural      0.0m       120,000
   6  Incoming Urban Typhoon   typhoon  urban      1.8m       200,000
========================================================================
```

*Figure 3. Demo run — top-2 blending, final solution, and case saved (6 cases)*

Run: disaster_cbr_dss --new

```
================================================================================
  DISASTER RESPONSE DSS  -  Case-Based Reasoning (CDRRMO planning tool)
================================================================================
Case base loaded from disaster_cases.csv: 6 cases
   #  Event                    Hazard   Setting   Depth  Pop. at risk
   1  Typhoon Ursula 2019      typhoon  coastal    0.5m        80,000
   2  Typhoon Odette 2021      typhoon  urban      1.2m       250,000
   3  Typhoon Kristine 2024    typhoon  urban      2.0m       300,000
   4  Habagat Flood 2023       flood    urban      1.5m       150,000
   5  Taal Volcano 2020        volcano  rural      0.0m       120,000
   6  Incoming Urban Typhoon   typhoon  urban      1.8m       200,000
Describe the new event:
   Event name: Habagat Surge 2026
   Hazard (typhoon/flood/volcano): flood
   Setting (urban/coastal/rural): urban
   Signal no. / alert level: 0
   Expected flood depth (m): 1.2
   Population at risk: 120000
   Hours before impact: 30
--------------------------------------------------------------------------------
NEW PROBLEM: Habagat Surge 2026
   hazard               : flood
   setting              : urban
   signal_or_magnitude  : 0.0
   flood_depth_m        : 1.2
   population_at_risk   : 120,000
   hours_before_impact  : 30
--------------------------------------------------------------------------------
SIMILARITY ASSESSMENT & RETRIEVAL
   Rank 1: Habagat Flood 2023       similarity = 0.964
   Rank 2: Typhoon Odette 2021      similarity = 0.729
   Rank 3: Incoming Urban Typhoon   similarity = 0.715
   Rank 4: Typhoon Kristine 2024    similarity = 0.660
   Rank 5: Typhoon Ursula 2019      similarity = 0.647
   Rank 6: Taal Volcano 2020        similarity = 0.357
   >> Retrieved: Habagat Flood 2023 and Typhoon Odette 2021
--------------------------------------------------------------------------------
CASE ADAPTATION (blend of top-2 cases)
   Blend weights: Habagat Flood 2023 = 0.57, Typhoon Odette 2021 = 0.43
   Severity index = 1.02
   evacuees       rate/1k:  233.33 &  248.00 ->  239.65 x 120k x 1.02 = 29,334
   shelters       rate/1k:    0.47 &    0.52 ->    0.49 x 120k x 1.02 = 60
   relief_packs   rate/1k:  700.00 &  744.00 ->  718.95 x 120k x 1.02 = 88,000
   rescue_teams   rate/1k:    0.20 &    0.16 ->    0.18 x 120k x 1.02 = 23
   medical_units  rate/1k:    0.07 &    0.08 ->    0.07 x 120k x 1.02 = 9
--------------------------------------------------------------------------------
```

*Figure 4. Second run (--new) — user types a monsoon flood event*

Run: disaster_cbr_dss --new (continued)

```
ADAPTED FINAL SOLUTION
   evacuees      : 29,334
   shelters      : 60
   relief_packs  : 88,000
   rescue_teams  : 23
   medical_units : 9
   strategy      : Boat rescue along riverbanks; gyms used as shelters; open schools and covered courts as
shelters
--------------------------------------------------------------------------------
LEARNING: New case saved -> 'Habagat Surge 2026' written to disaster_cases.csv
Updated case base (7 cases):
   #  Event                    Hazard   Setting   Depth  Pop. at risk
   1  Typhoon Ursula 2019      typhoon  coastal    0.5m        80,000
   2  Typhoon Odette 2021      typhoon  urban      1.2m       250,000
   3  Typhoon Kristine 2024    typhoon  urban      2.0m       300,000
   4  Habagat Flood 2023       flood    urban      1.5m       150,000
   5  Taal Volcano 2020        volcano  rural      0.0m       120,000
   6  Incoming Urban Typhoon   typhoon  urban      1.8m       200,000
   7  Habagat Surge 2026       flood    urban      1.2m       120,000
================================================================================
```

*Figure 5. Second run — adapted plan and case base updated to 7 cases*

## C. Pair Reflection

Ingal worked on the case representation, the CSV loading and saving, and the similarity tables, while Bautista worked on the Gaussian similarity, the top-2 blending adaptation, and the documentation, and we tested the program

together. One challenge was that copying only one case felt too rigid, so we tried blending the top two cases, which needed converting resources into per-1,000-people rates first. We also had to fix the strategy text because the rules sometimes repeated steps that the past case already had. Through this activity, we learned that CBR solves problems the way people do, by remembering similar experiences and adjusting them. We also saw that the retain step really matters, because the event we saved in the first run appeared in the ranking of the second run.